

Master : Cryptologie et Sécurité Informatique (CSI)

Rapport

DÉTECTION D'URL MALVEILLANTES À L'AIDE DE L'APPRENTISSAGE AUTOMATIQUE

Réalisé par :

DAOUAYRY Youssef

Encadré par :

Pr. SOUFI Adil

Année Scolaire : 2024/2025

TABLE DES MATIÈRES

Introduction	3
1 Uniform Ressource Locator	4
1.1 Définition	4
1.2 Composants d'une URL	4
1.3 Utilisation Malveillante des URLs	5
2 Prétraitement des Données et Entraînement des Modèles	6
2.1 Description du jeu de données	6
2.2 Prétraitement des Données	7
2.3 Extraction des caractéristiques (Feature Extraction)	8
2.4 Entraînement des Modèles :	12
2.4.1 Division des Données	13
2.5 Évaluation des Modèles	14
2.6 Cross-Validation	15
2.7 Hyperparameter Tuning	16
3 Intégration du Modèle dans une Application Web	19
Conclusion	22
Répertoire GitHub	23
Références	23

INTRODUCTION

Des études ont montré qu'au moins 47% de la population mondiale est connectée à un moment donné . La plupart des utilisateurs, en particulier ceux qui utilisent des ordinateurs personnels, sont beaucoup plus vulnérables aux attaques et menaces externes car ils ont très peu de connaissances sur certaines des attaques qui peuvent être lancées contre eux et sur la manière de se protéger. La prolifération des activités malveillantes sur Internet représente une menace significative pour la cybersécurité. Parmi les différents vecteurs d'attaque, les URL malveillantes constituent une méthode courante utilisée par les acteurs de la menace pour distribuer des logiciels malveillants, mener des attaques de phishing et compromettre la sécurité des utilisateurs. Ainsi, la capacité à classifier avec précision les URL comme étant bénignes ou malveillantes est cruciale pour protéger les utilisateurs et les organisations contre les cybermenaces potentielles.

L'idée centrale de ce projet tourne autour de la protection des utilisateurs ordinaires contre les URL malveillantes, dans le but de développer un système basé sur l'apprentissage automatique pour la détection automatique des URL malveillantes. En exploitant les caractéristiques extraites des URL, nous cherchons à créer des modèles prédictifs capables de distinguer avec précision entre les URL bénignes et malveillantes.

La pertinence de ce projet réside dans son potentiel à renforcer les mesures de cybersécurité en fournissant des outils automatisés pour la détection proactive des menaces et leur atténuation. En identifiant efficacement les URL malveillantes, les utilisateurs peuvent être alertés des risques potentiels pour la sécurité, et les organisations peuvent mettre en œuvre des contre-mesures appropriées pour protéger leurs systèmes et leurs données.

CHAPITRE 1

UNIFORM RESSOURCE LOCATOR

Les URL, ou Uniform Resource Locators, constituent le pilier de l'internet, agissant comme les adresses numériques qui guident les utilisateurs vers diverses ressources en ligne. Comprendre les URL est primordial pour appréhender les subtilités de la navigation web et de la récupération d'informations.

1.1 Définition

Au cœur de leur fonctionnement, une URL est une chaîne de caractères qui identifie de manière unique une ressource sur l'internet. Elle englobe l'adresse d'une page web, d'un fichier, d'une image ou de toute autre entité numérique accessible via le web. Une URL se compose généralement de plusieurs composants, chacun servant un but spécifique dans l'identification et la localisation de la ressource souhaitée.

1.2 Composants d'une URL

Une URL est une chaîne de caractères structurée qui permet d'identifier et de localiser une ressource spécifique sur l'internet. Chaque composant d'une URL joue un rôle précis dans sa fonctionnalité globale, contribuant à sa capacité à diriger les utilisateurs vers des pages web, des fichiers, des images, des vidéos et d'autres contenus numériques.



FIGURE 1.1 – structure d'une url

Une URL se décompose en plusieurs parties qui sont :

- **Protocole** : Le protocole spécifie la méthode utilisée pour accéder à la ressource. Les protocoles les plus courants sont "http://" pour les sites web non sécurisés et "https://" pour les sites web sécurisés utilisant le chiffrement SSL/TLS.
- **Nom de Domaine** : Le nom de domaine identifie le serveur sur lequel la ressource est hébergée. Il peut être composé d'un sous-domaine (facultatif), du nom de domaine de premier niveau (TLD) et du domaine de deuxième niveau (SLD). Par exemple, dans "www.example.com", "www" est le sous-domaine, "example" est le domaine de deuxième niveau et "com" est le TLD.
- **Chemin** : Le chemin spécifie l'emplacement précis de la ressource sur le serveur. Il peut inclure des répertoires et des sous-répertoires qui organisent les fichiers sur le serveur. Par exemple, dans "www.example.com/page1/page2", "page1/page2" est le chemin vers la ressource.
- **Paramètres (optionnels)** : Les paramètres peuvent être utilisés pour transmettre des informations supplémentaires à la ressource. Ils sont généralement séparés du reste de l'URL par un point d'interrogation ("?") et peuvent prendre la forme de paires clé-valeur. Par exemple, dans "www.example.com/page?id=123", "id" est le paramètre et "123" est sa valeur.
- **Fragment (optionnel)** : Le fragment spécifie une section spécifique de la ressource à afficher. Il est généralement indiqué par un signe dièse("#") suivi d'une ancre ou d'un identifiant. Par exemple, dans "www.example.com/pagesection1", "section1" est le fragment.

1.3 Utilisation Malveillante des URLs

Les URL peuvent être utilisées comme vecteurs pour diffuser des logiciels malveillants ou pour des attaques de phishing, où des utilisateurs sont trompés pour divulguer des informations sensibles telles que des identifiants de connexion ou des informations financières. Les méthodes courantes incluent la dissimulation de liens malveillants derrière des URL légitimes, la création de faux sites Web qui imitent des pages de connexion authentiques pour voler des informations d'identification, ou l'utilisation de raccourcisseurs d'URL pour masquer des destinations malveillantes. Les cybercriminels exploitent souvent la confiance des utilisateurs dans les URL pour les inciter à cliquer sur des liens dangereux.

Maintenant, passons à la prochaine section qui aborde l'implémentation de l'apprentissage automatique pour détecter ces URL malveillantes.

CHAPITRE 2

PRÉTRAITEMENT DES DONNÉES ET ENTRAÎNEMENT DES MODÈLES

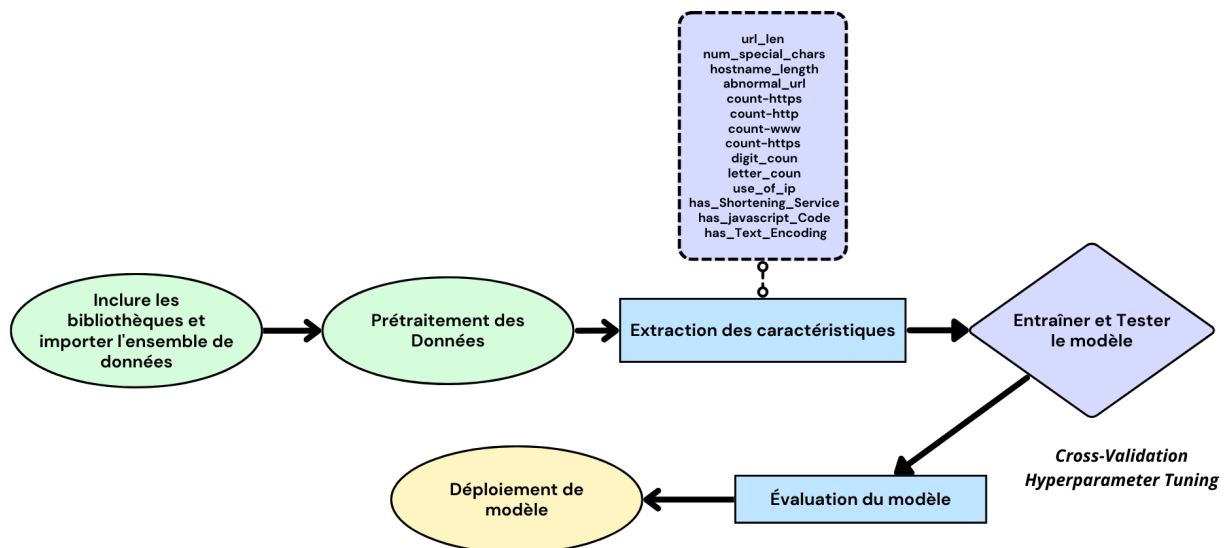


FIGURE 2.1 – Architecture

2.1 Description du jeu de données

Le jeu de données utilisé dans ce projet a été obtenu à partir de Kaggle, une plateforme connue pour héberger une large gamme de jeux de données pour les projets d'apprentissage automatique et de science des données. Ce jeu de données se compose de deux colonnes principales : "url" et "label". La colonne "url" contient les URL elles-mêmes, tandis que la colonne "label" indique la classification de chaque URL comme étant bénigne ou malveillante.

Au total, le jeu de données comprend plus de 650 000 lignes, ce qui en fait une ressource substantielle pour l'entraînement et l'évaluation des modèles d'apprentissage automatique.

Le grand volume de données permet un développement et une validation robustes des modèles, offrant de nombreuses opportunités pour explorer les motifs et les relations au sein du jeu de données.

	url	label
0	br-icloud.com.br	malicious
1	mp3raid.com/music/krizz_kaliko.html	benign
2	bopsecrets.org/rexroth/cr/1.htm	benign
3	http://www.garage-pirene.be/index.php?option=...	malicious
4	http://adventure-nicaragua.net/index.php?optio...	malicious
5	http://buzzfil.net/m/show-art/ils-etaient-loin...	benign
6	espn.go.com/nba/player/_/id/3457/brandon-rush	benign
7	yourbittorrent.com/?q=anthony-hamilton-soulife	benign
8	http://www.pashminaonline.com/pure-pashminas	malicious
9	allmusic.com/album/crazy-from-the-heat-r16990	benign

FIGURE 2.2 – Example from dataset

2.2 Prétraitement des Données

Le jeu de données a été soumis à plusieurs étapes de prétraitement pour standardiser le format, éliminer le bruit et créer un ensemble de données bien structuré pour l'entraînement et l'évaluation des modèles. Les étapes suivantes sont appliquées :

Éliminer les valeurs nulles :

Aucune valeur nulle n'a été trouvée dans les données, assurant ainsi l'intégrité des observations utilisées dans l'analyse subséquente.

```
data.isnull().sum()
```

```
url      0
label    0
dtype: int64
```

FIGURE 2.3 – les valeurs nulles

Supprimer les doublons :

Les doublons ont été identifiés et supprimés. Pour les doublons présentant des étiquettes différentes, les deux instances ont été retirées pour éviter toute ambiguïté dans l'étiquetage des données. Pour les doublons ayant la même étiquette, une seule instance a été conservée pour maintenir la diversité des données tout en éliminant les duplications.

```
duplicate_urls = data[data.duplicated(subset=['url'], keep=False)]
len(duplicate_urls)
```

```
12279
```

Remove duplicates with the same label, keeping one instance:

```
data = data.drop_duplicates(subset=['url', 'label'], keep='first')
```

Now we only have duplicates with different labels , so we will remove both instances:

```
duplicate_urls = data[data.duplicated(subset=['url'], keep=False)]
indices_to_drop = duplicate_urls.index
data = data.drop(indices_to_drop)
```

```
data.shape
```

```
(641125, 2)
```

FIGURE 2.4 – urls doublons

L'encodage :

Pour uniformiser le format des étiquettes, les étiquettes ont été transformées en valeurs numériques à l'aide d'une fonction de mappage définie préalablement, les étiquettes "benign" ont été codées comme 0 et les étiquettes "malicious" comme 1, ce qui a simplifié le processus d'analyse et de modélisation subséquentes.

```
label_mapping = {"benign": 0, "malicious": 1}
data['label'] = data['label'].map(label_mapping)
```

FIGURE 2.5 – label mapping

2.3 Extraction des caractéristiques (Feature Extraction)

Après la transformation et le nettoyage des données, l'étape suivante consiste à l'extraction des caractéristiques . Dans cette section, nous décrivons les caractéristiques extraites à partir des URL, qui sont utilisées comme entrées pour notre modèle de détection de URL malveillants.

- **Longueur de l'URL (url_len)** : Cette caractéristique représente la longueur totale de l'URL, mesurée en nombre de caractères. Les URL plus longues peuvent parfois être associées à des comportements suspects.
- **Nombre de caractères spéciaux (num_special_chars)** : Cela compte le nombre de caractères spéciaux présents dans l'URL, tels que les tirets, les underscores, les points d'interrogation, etc. Les caractères spéciaux inhabituels peuvent indiquer un lien potentiellement malveillant.
- **Longueur du nom d'hôte (hostname_length)** : Cette caractéristique mesure la longueur du nom d'hôte extrait de l'URL. Les noms d'hôtes plus longs ou plus courts

peuvent être associés à différents types de sites web.

- **URL anormale (abnormal_url)** : Cette caractéristique indique si l'URL présente des schémas inhabituels ou des anomalies dans sa structure. Les URL anormales peuvent être indicatives de tentatives de phishing ou de redirections malveillantes.
- **Nombre d'occurrences de "https" (count-https)** : Cela compte le nombre de fois où le préfixe "https" apparaît dans l'URL. Les connexions sécurisées sont généralement associées à des sites web légitimes.
- **Nombre d'occurrences de "http" (count-http)** : De manière similaire, cette caractéristique compte le nombre de fois où le préfixe "http" apparaît dans l'URL. Les sites web non sécurisés peuvent être associés à des activités malveillantes.
- **Nombre d'occurrences de "www" (count-www)** : Cette caractéristique compte le nombre de fois où le préfixe "www" apparaît dans l'URL. La présence de "www" peut être un indicateur de la structure de l'URL.
- **Nombre de chiffres (digit_count)** : Cette caractéristique compte le nombre de chiffres présents dans l'URL. Les URL contenant de nombreux chiffres peuvent être associées à des URL de phishing ou à des sites web frauduleux.
- **Nombre de lettres (letter_count)** : De manière similaire, cette caractéristique compte le nombre de lettres présentes dans l'URL. Les URL avec un contenu principalement composé de lettres peuvent être associées à des sites web légitimes.
- **Utilisation d'une adresse IP (use_of_ip)** : Cette caractéristique indique si l'URL utilise une adresse IP plutôt qu'un nom de domaine. Les adresses IP directes peuvent être utilisées dans des URL malveillantes pour cacher la véritable destination du lien.
- **Présence d'un service de raccourcissement d'URL (has_Shortening_Service)** : Cette caractéristique indique si l'URL utilise un service de raccourcissement d'URL comme Bit.ly ou TinyURL. Les services de raccourcissement d'URL sont parfois utilisés pour masquer des URL malveillantes.
- **Présence de code JavaScript (has_javascript_Code)** : Cette caractéristique indique si l'URL contient du code JavaScript. Les scripts JavaScript peuvent être utilisés pour rediriger les utilisateurs vers des sites web malveillants ou pour exécuter des attaques basées sur le navigateur.
- **Présence de codage de texte (has_Text_Encoding)** : Enfin, cette caractéristique indique si l'URL utilise un codage de texte particulier, tel que UTF-8. Les techniques d'encodage peuvent parfois être utilisées pour masquer des URL malveillantes.

Ces caractéristiques fournissent une représentation diversifiée des URL et de leurs attributs associés, ce qui permet à notre modèle d'apprentissage automatique de détecter efficacement les logiciels malveillants.

	url	label	url_len	num_special_chars	hostname_length	abnormal_url	count-https	count-http	count-www	digit_count	letter_cou
0	br-icloud.com.br	1	16	3	0	0	0	0	0	0	
1	mp3raid.com/music/krizz_kaliko.html	0	35	2	0	0	0	0	0	1	
2	bopsecrets.org/rexroth/cr/1.htm	0	31	2	0	0	0	0	0	1	
3	http://www.garage-pirenne.be/index.php?option=...	1	88	10	21	1	0	1	1	7	
4	http://adventure-nicaragua.net/index.php?option=...	1	235	8	23	1	0	1	0	22	1
5	http://buzzfil.net/m/show-art/ils-etaient-loin...	0	118	19	11	1	0	1	0	1	
6	espn.go.com/nba/player/_/id/3457/brandon-rush	0	45	3	0	0	0	0	0	4	
7	yourbittorrent.com/?q=anthony-hamilton-soulife	0	46	5	0	0	0	0	0	0	
8	http://www.pashminaonline.com/pure-pashminas	1	44	4	22	1	0	1	1	0	
9	allmusic.com/album/crazy-from-the-heat-r16990	0	45	5	0	0	0	0	0	5	

FIGURE 2.6 – Example from dataset

À propos du jeu de données :

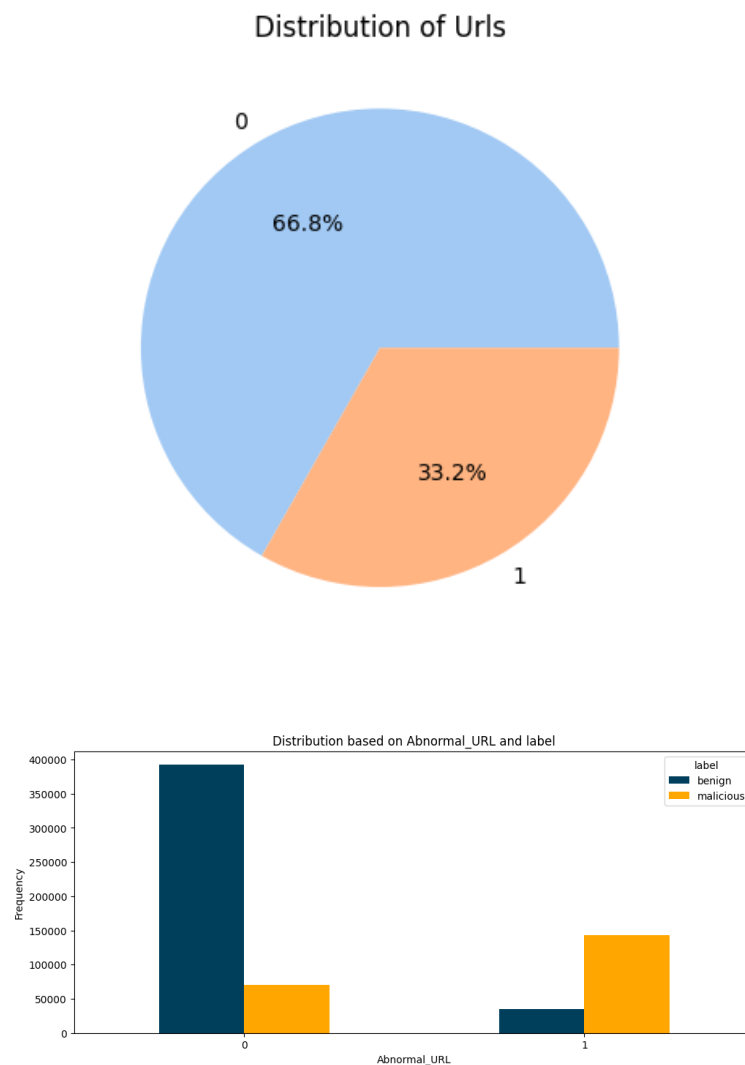
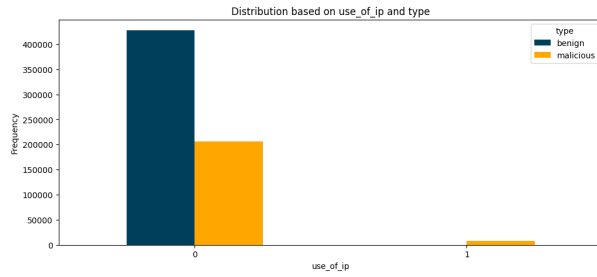
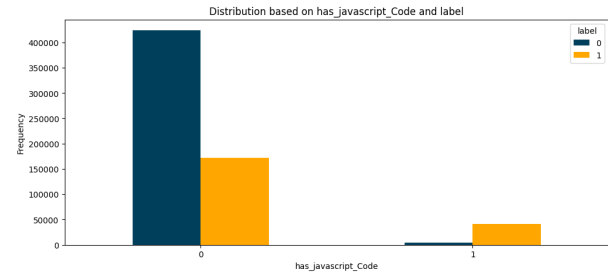


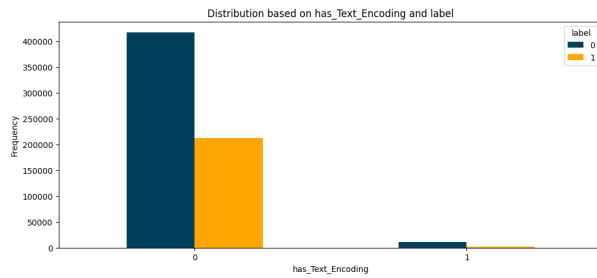
FIGURE 2.7 – abnormal_url



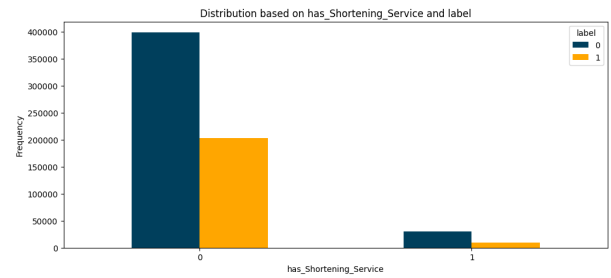
(a) use_of_ip



(b) has_javascript_Code



(c) (has_Text_Encodingr



(d) has_Shortening_Service

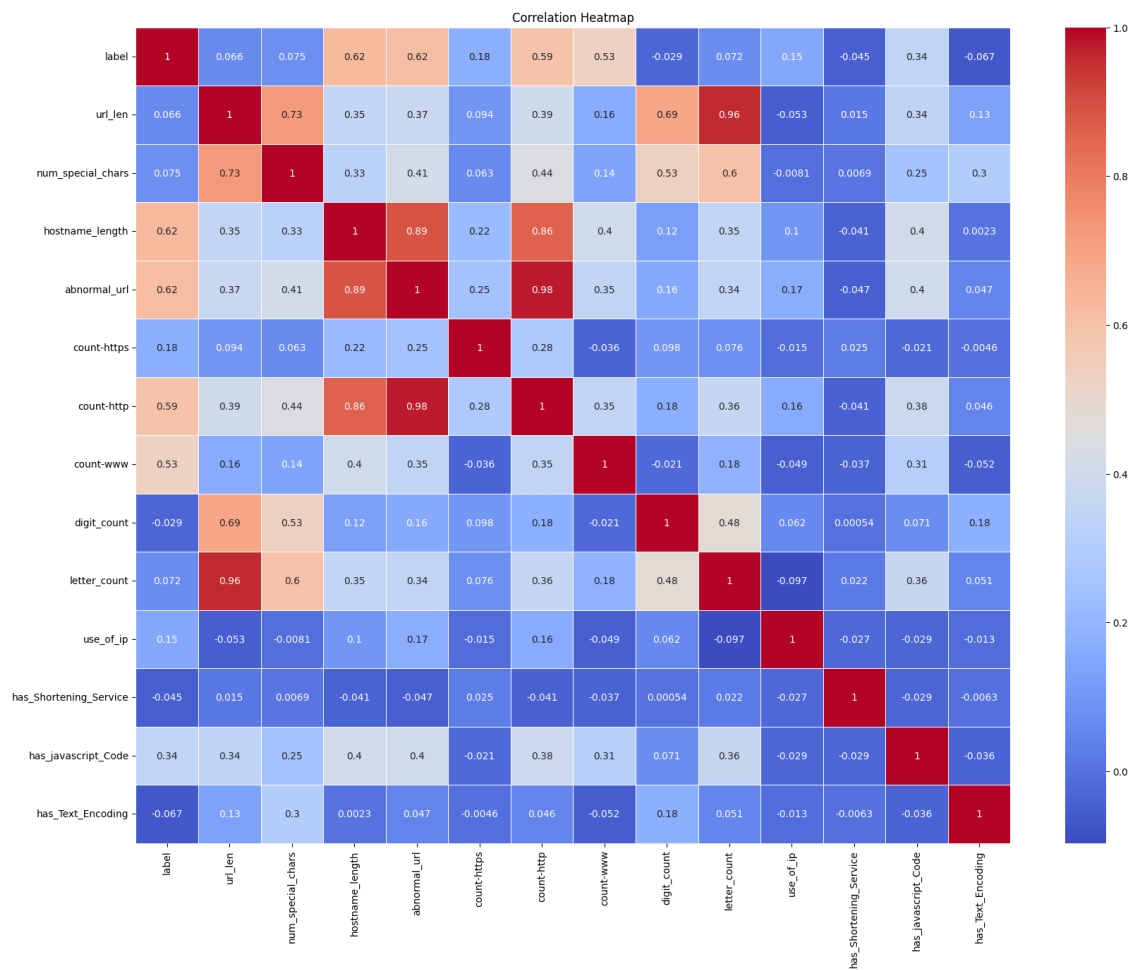


FIGURE 2.9 – correlation

2.4 Entraînement des Modèles :

Dans cette section, nous plongeons dans le processus d'entraînement des modèles d'apprentissage automatique pour détecter les URL malveillantes. L'objectif est de développer des classificateurs robustes capables de distinguer avec précision les URL bénignes des URL malveillantes en fonction de diverses caractéristiques extraites des données. La phase d'entraînement joue un rôle crucial dans la réalisation de cet objectif en exploitant des techniques d'apprentissage supervisé pour apprendre des modèles à partir de données étiquetées.

Nous avons envisagé plusieurs algorithmes d'apprentissage automatique connus pour leur efficacité dans les tâches de classification binaire. Les modèles sélectionnés comprennent :

- **Logistic Regression** : La régression logistique est un algorithme largement utilisé pour les tâches de classification binaire. Elle modélise la probabilité d'un résultat binaire en utilisant une fonction logistique, ce qui la rend adaptée pour prédire la probabilité qu'une URL soit bénigne ou malveillante.

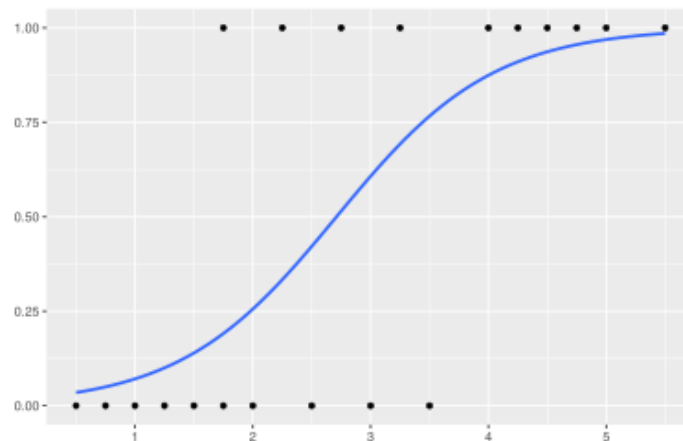


FIGURE 2.10 – Logistic Regression

- **K-Nearest Neighbors** : Le classificateur des k plus proches voisins est une méthode non paramétrique utilisée pour les tâches de classification et de régression. Il attribue la classe d'une URL en fonction de la classe majoritaire parmi ses k voisins les plus proches dans l'espace des caractéristiques.

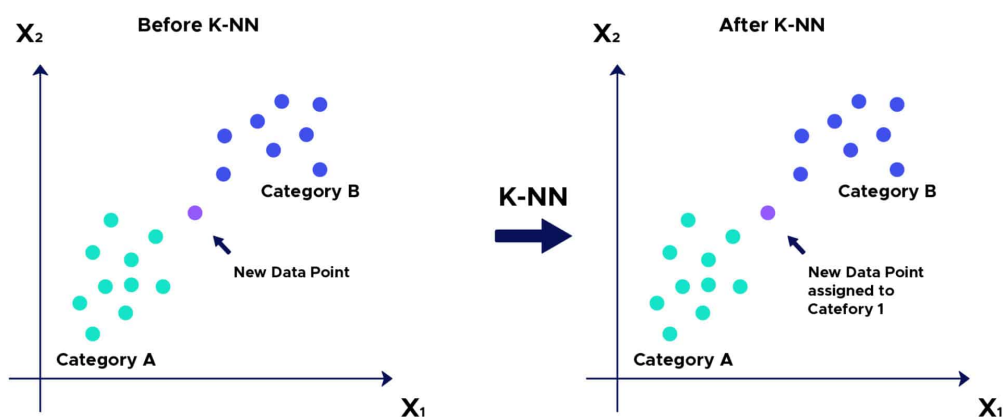


FIGURE 2.11 – KNN

- **Decision Tree Classifier** : Le classificateur par arbre de décision est un modèle structuré en arbre qui divise les données en sous-ensembles en fonction de la valeur des caractéristiques d'entrée. Il est capable de traiter à la fois les données numériques et catégorielles, ce qui le rend adapté aux tâches de classification des URL.

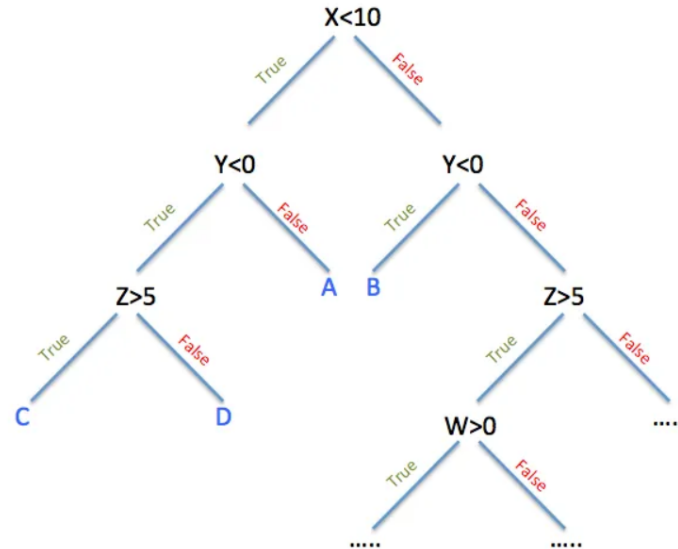


FIGURE 2.12 – Decision Tree

- **Random Forest Classifier** : Le classificateur par forêt aléatoire est une méthode d'apprentissage par ensemble qui combine plusieurs arbres de décision pour améliorer la généralisation et réduire le surajustement. Il fonctionne en agrégeant les prédictions de plusieurs arbres de décision, ce qui donne un classificateur plus robuste et précis.

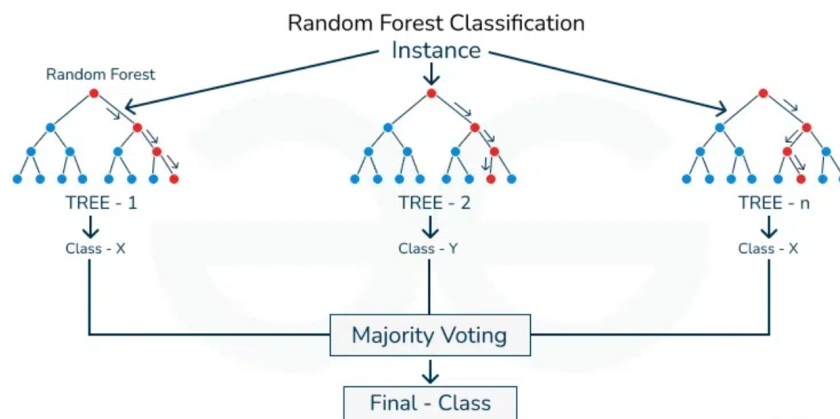


FIGURE 2.13 – Random Forest

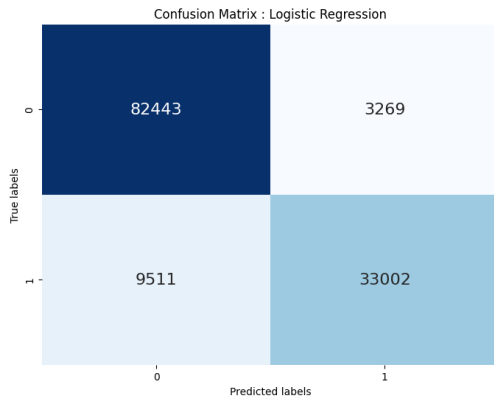
2.4.1 Division des Données

Nous divisons notre ensemble de données en un ensemble d'entraînement, représentant 80% des données, et un ensemble de test, contenant les 20% restants. Cette division nous permet d'évaluer les performances de nos modèles de manière impartiale et de prévenir le surapprentissage, assurant ainsi une généralisation robuste.

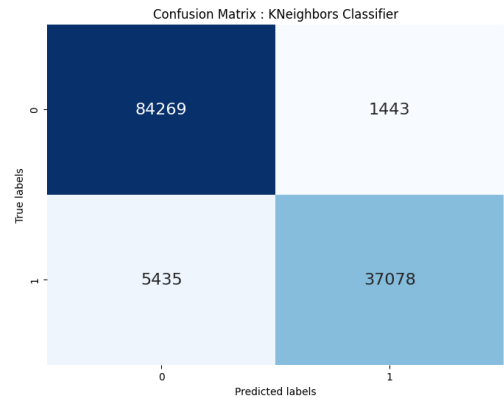
```
X = data.drop(columns=['url', 'label'])
y = data['label']
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size = 0.2, shuffle=True, random_state=5)
```

FIGURE 2.14 – train_test_split

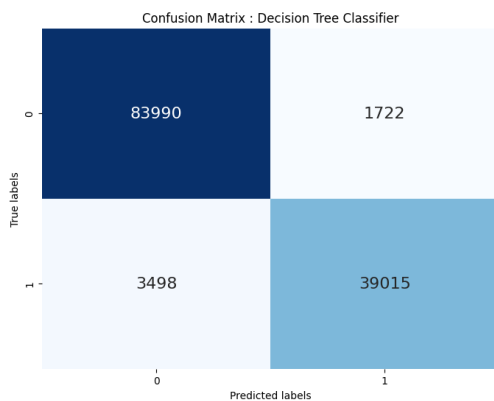
2.5 Évaluation des Modèles



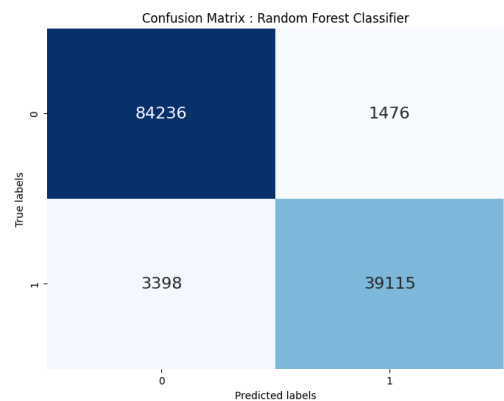
(a) Logistic Regression



(b) K-Nearest Neighbors



(c) Decision Tree Classifier



(d) Random Forest Classifie

FIGURE 2.15 – Métriques de Confusion

Model	Evaluation Metrics			
	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.9030	0.90	0.91	0.90
k-Nearest Neighbors	0.9464	0.94	0.96	0.95
Decision Tree	0.9593	0.96	0.96	0.96
Random Forest	0.9620	0.96	0.96	0.96

TABLE 2.1 – Model Evaluation Metrics

Analyse et discussion :

Les résultats montrent que tous les modèles ont bien performé dans la détection des URL malveillantes. Le modèle Random Forest a obtenu la meilleure précision globale (96.20%), suivi par le Decision Tree (95.93%), le k-Nearest Neighbors (94.64%), et enfin la régression logistique (90.03%). Bien que tous les modèles aient eu de bons résultats, Random Forest s'est

démarqué avec une sensibilité élevée et un bon équilibre entre précision et rappel. Il serait donc judicieux de considérer Random Forest comme le choix principal pour la détection des URL malveillantes.

2.6 Cross-Validation

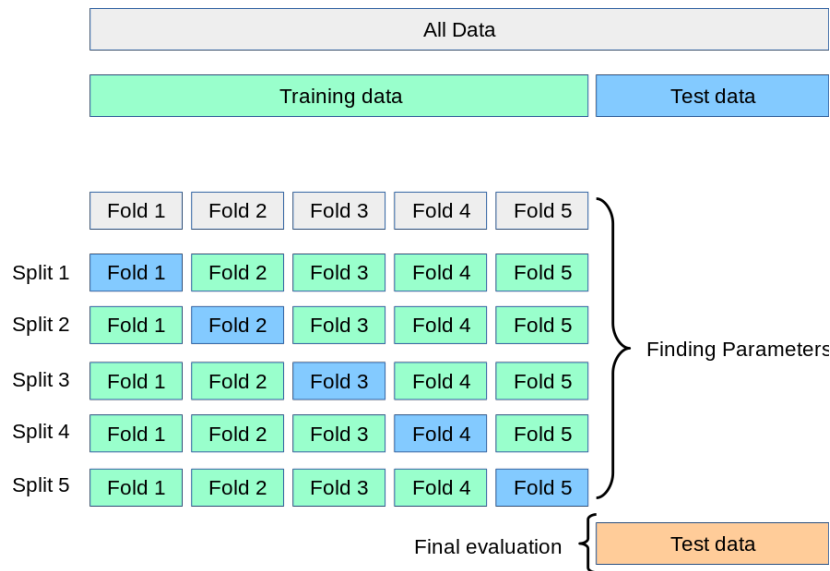


FIGURE 2.16 – Cross-Validation

Cross-Validation est une technique essentielle en apprentissage automatique qui permet d'évaluer la performance d'un modèle de manière robuste et fiable. Contrairement à une simple division des données en ensembles d'entraînement et de test, Cross-Validation utilise l'ensemble de données dans son intégralité pour évaluer la capacité de généralisation du modèle. Cette méthode est particulièrement utile lorsque l'ensemble de données est limité ou lorsque l'on souhaite obtenir une estimation plus précise de la performance du modèle.

Les résultats sont fournis ci-dessous :

TABLE 2.2 – Cross-Validation Scores for Different Models

Model	Mean Cross-Validation Score	Standard Deviation of Cross-Validation Score
Logistic Regression	0.901	0.0007
k-Nearest Neighbors	0.929	0.0014
Decision Tree	0.960	0.0004
Random Forest	0.963	0.0003

- **Mean Cross-Validation Score (Score Moyen de Validation Croisée) :** C'est la moyenne des performances du modèle sur plusieurs sous-ensembles de données lors de la validation croisée. Il représente la performance globale du modèle.
- **Standard Deviation of Cross-Validation Score (Écart-Type du Score de Validation Croisée) :** C'est une mesure de la variabilité des performances du modèle sur différents sous-ensembles de données lors de la validation croisée. Un écart-type plus faible indique une stabilité dans les performances du modèle.

Analyse et discussion :

Les résultats de Cross-Validation confirment la performance des modèles évalués dans la détection des URL malveillantes. Le score moyen de validation croisée pour la régression logistique est de 0.90, suivi de près par les méthodes k-Nearest Neighbors (0.93), Decision Tree (0.96) et Random Forest (0.96). Les scores moyens élevés indiquent la capacité des modèles à généraliser efficacement sur différentes partitions des données.

De plus, les écarts-types relativement faibles des scores de validation croisée suggèrent une stabilité dans les performances des modèles sur différents sous-ensembles de données. Cela renforce la confiance dans la fiabilité des estimations de performance obtenues par la validation croisée.

Ces résultats confirment la robustesse des modèles évalués et fournissent des indications supplémentaires sur leur capacité de généralisation dans des scénarios pratiques.

2.7 Hyperparameter Tuning

Hyperparameter Tuning consiste à trouver le meilleur ensemble d'hyperparamètres pour votre modèle d'apprentissage automatique afin d'optimiser sa performance. Les hyperparamètres sont des paramètres qui sont définis avant le début du processus d'apprentissage. Des techniques comme *GridSearchCV* et *RandomSearchCV* sont couramment utilisées pour l'optimisation des hyperparamètres, dans notre cas, nous utiliserons *GridSearchCV*

Ci-dessous, nous présentons les paramètres principaux de chaque modèle utilisé dans notre étude :

- **Logistic Regression :**

penalty : Paramètre contrôlant la régularisation

C : Force de régularisation. plus la valeur de C est petite, plus la régularisation est forte.

Hyperparamètres	Différentes Valeurs
C	0.001, 0.01, 0.1, 1, 10, 100
Pénalisation	L1, L2

Résultats :

Hyperparamètre	Valeur
C	10
Pénalisation	L2

Logistic Regression - After Hyperparameter Tuning:

	precision	recall	f1-score	support
benign	0.90	0.96	0.93	85712
malicious	0.91	0.78	0.84	42513
accuracy			0.90	128225
macro avg	0.90	0.87	0.88	128225
weighted avg	0.90	0.90	0.90	128225

- **K-Nearest Neighbors :**

n_neighbors : Nombre de voisins à considérer lors de la prédiction.

Résultats :

Hyperparamètres	Différentes Valeurs
n_neighbors	3, 5, 7, 10 ,20 , 30

Hyperparamètre	Valeur
n_neighbors	5

K-Nearest Neighbors - After Hyperparameter Tuning:				
	precision	recall	f1-score	support
benign	0.95	0.98	0.97	85712
malicious	0.96	0.90	0.93	42513
accuracy			0.95	128225
macro avg	0.95	0.94	0.95	128225
weighted avg	0.95	0.95	0.95	128225

- **Decision Tree Classifier :**

Criterion : Métrique utilisée pour mesurer la qualité de la division.

max_depth : Profondeur maximale de l'arbre.

min_samples_split : Nombre minimum d'échantillons requis pour diviser un nœud interne.

Hyperparamètres	Différentes Valeurs
Criterion	gini, entropy
max_depth	None, 5, 10, 15
min_samples_split	2, 5, 10

Résultats :

Hyperparamètres	Valeur
Criterion	entropy
max_depth	None
min_samples_split	2

Decision Tree - After Hyperparameter Tuning:				
	precision	recall	f1-score	support
benign	0.96	0.98	0.97	85712
malicious	0.96	0.92	0.94	42513
accuracy			0.96	128225
macro avg	0.96	0.95	0.95	128225
weighted avg	0.96	0.96	0.96	128225

- **Random Forest Classifier :**

n_estimators : Nombre d'arbres dans la forêt aléatoire.

max_depth : Profondeur maximale de chaque arbre de décision.

max_leaf_nodes : Nombre maximum de nœuds feuilles dans chaque arbre..

Hyperparamètre	Différentes Valeurs
n_estimators	100, 200, 300
max_depth	None, 10, 20
max_leaf_nodes	None, 3, 6

Résultats :

Hyperparamètre	Valeur
n_estimators	100
max_depth	None
max_leaf_nodes	None

Random Forest - After Hyperparameter Tuning:				
	precision	recall	f1-score	support
benign	0.96	0.98	0.97	85712
malicious	0.96	0.92	0.94	42513
accuracy			0.96	128225
macro avg	0.96	0.95	0.96	128225
weighted avg	0.96	0.96	0.96	128225

Analyse et discussion :

Après l'optimisation des hyperparamètres, les performances des modèles ont globalement maintenu leur stabilité, avec peu de changements significatifs. Logistic Regression et KNN ont montré des variations mineures dans leurs métriques de performance, suggérant que les valeurs par défaut des hyperparamètres étaient déjà bien adaptées à notre ensemble de données.

Decision Tree Classifier et Random Forest Classifier ont également conservé des performances stables. Bien que des ajustements aient été apportés lors de l'optimisation des hyperparamètres, les améliorations dans les métriques de performance étaient marginales, confirmant que les valeurs par défaut étaient déjà efficaces.

Notamment, Random Forest est restée le meilleur modèle

CHAPITRE 3

INTÉGRATION DU MODÈLE DANS UNE APPLICATION WEB

Dans le processus de création de l'application, j'ai utilisé Flask pour le backend et HTML, CSS et JavaScript pour le frontend. Flask, en tant que framework Web minimaliste et flexible en Python, a été choisi pour sa simplicité et sa facilité d'utilisation dans la création de l'API de l'application. Il fournit les fonctionnalités nécessaires pour créer rapidement des applications Web robustes. D'autre part, HTML, CSS et JavaScript ont été utilisés pour concevoir l'interface utilisateur, offrant ainsi une expérience utilisateur fluide et interactive. Ces technologies ont permis de créer une application Web moderne et intuitive, permettant aux utilisateurs d'interagir facilement avec le système de détection d'URL malveillantes.

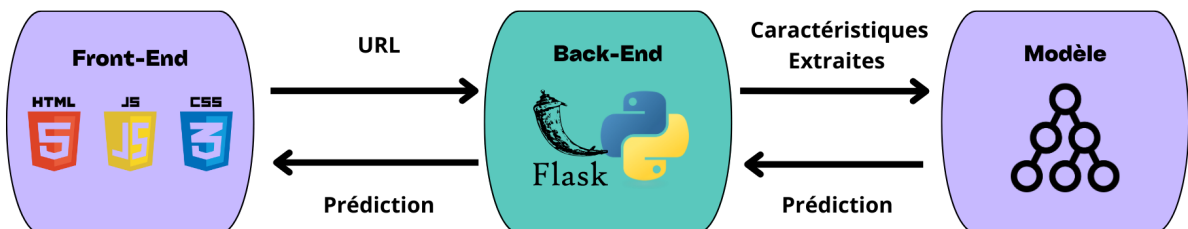
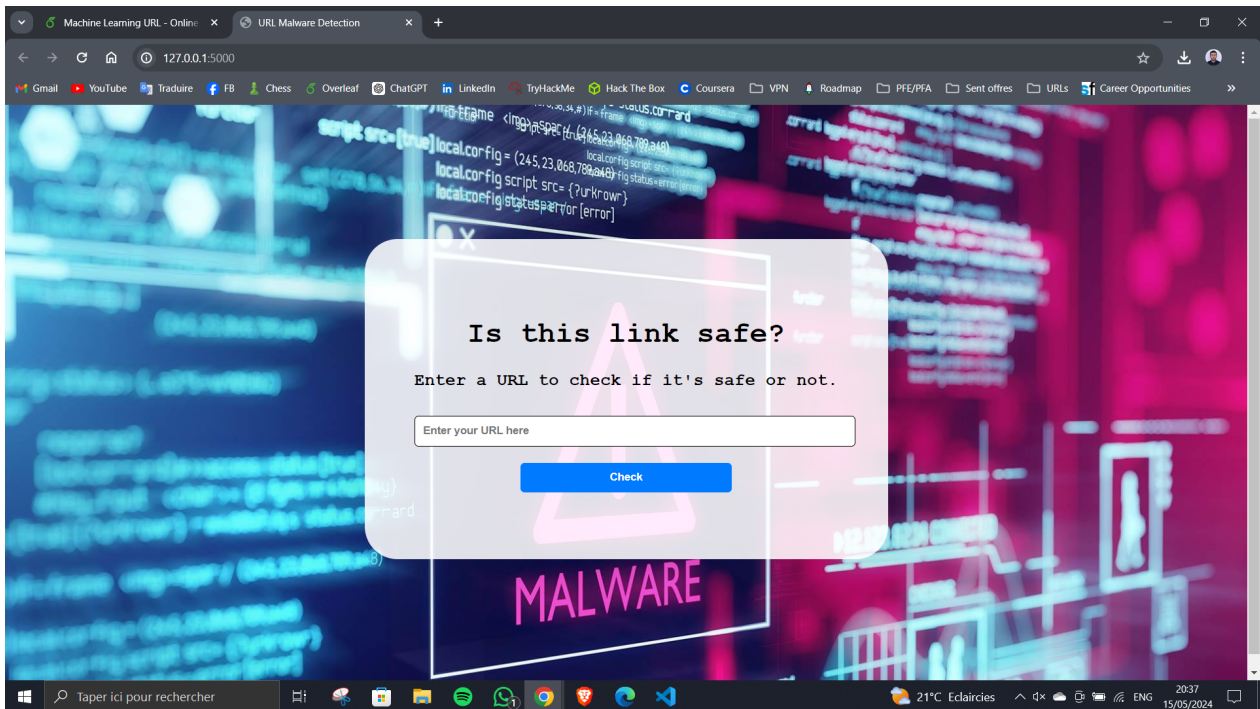
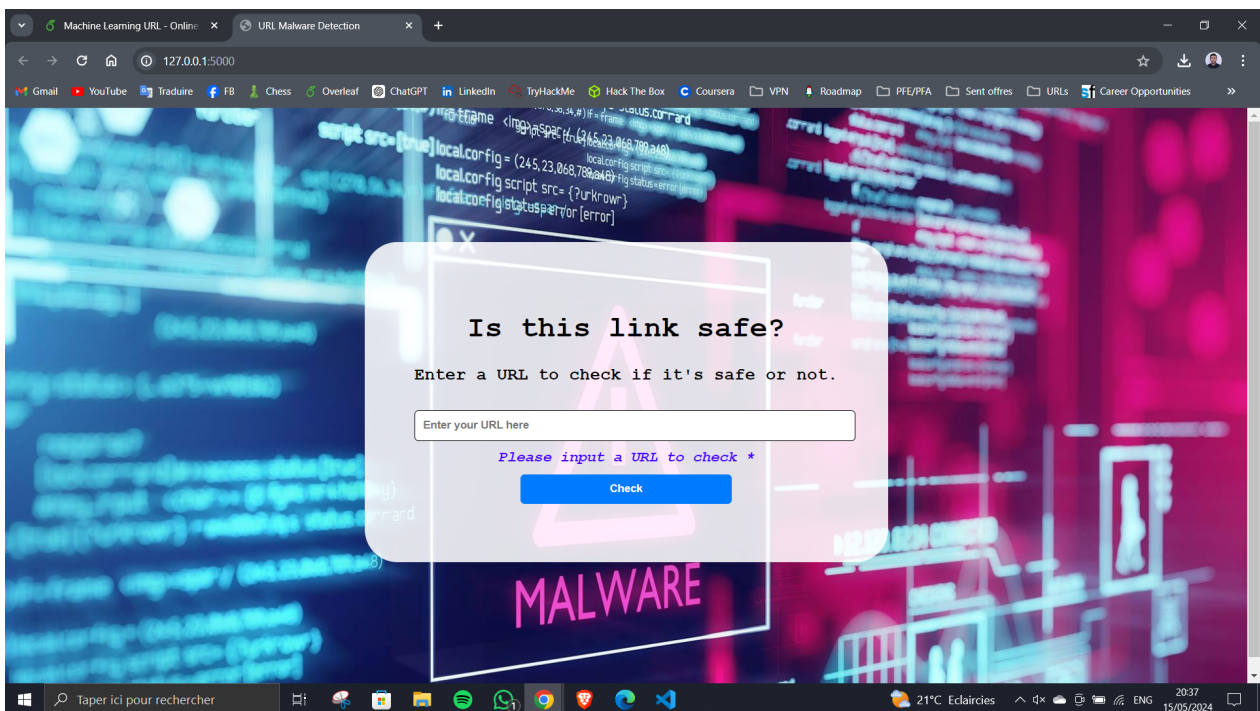


FIGURE 3.1 – Web app architecture

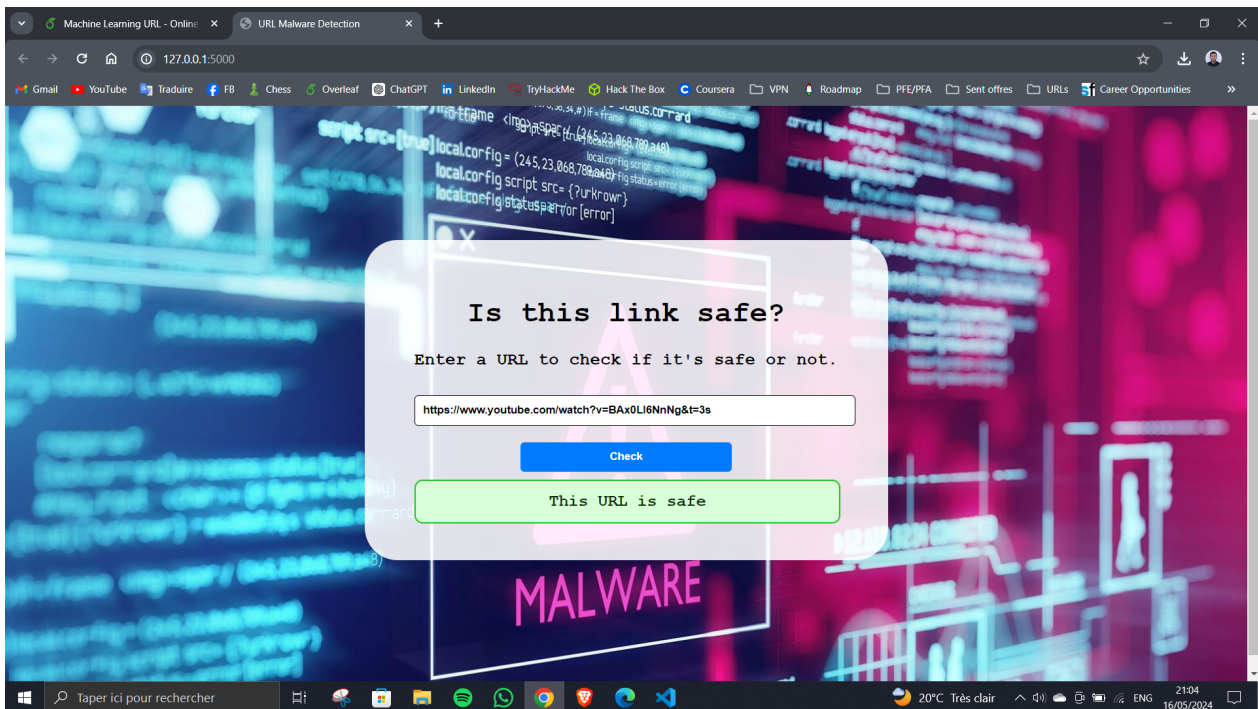
L'application Web est conçue avec une interface conviviale, guidant l'utilisateur à travers le processus de saisie pour garantir des prédictions précises et significatives. Les utilisateurs peuvent facilement saisir l'URL, et l'application communique de manière transparente avec le modèle Random Forest entraîné pour générer une prédiction si l'URL est sécurisée ou non.



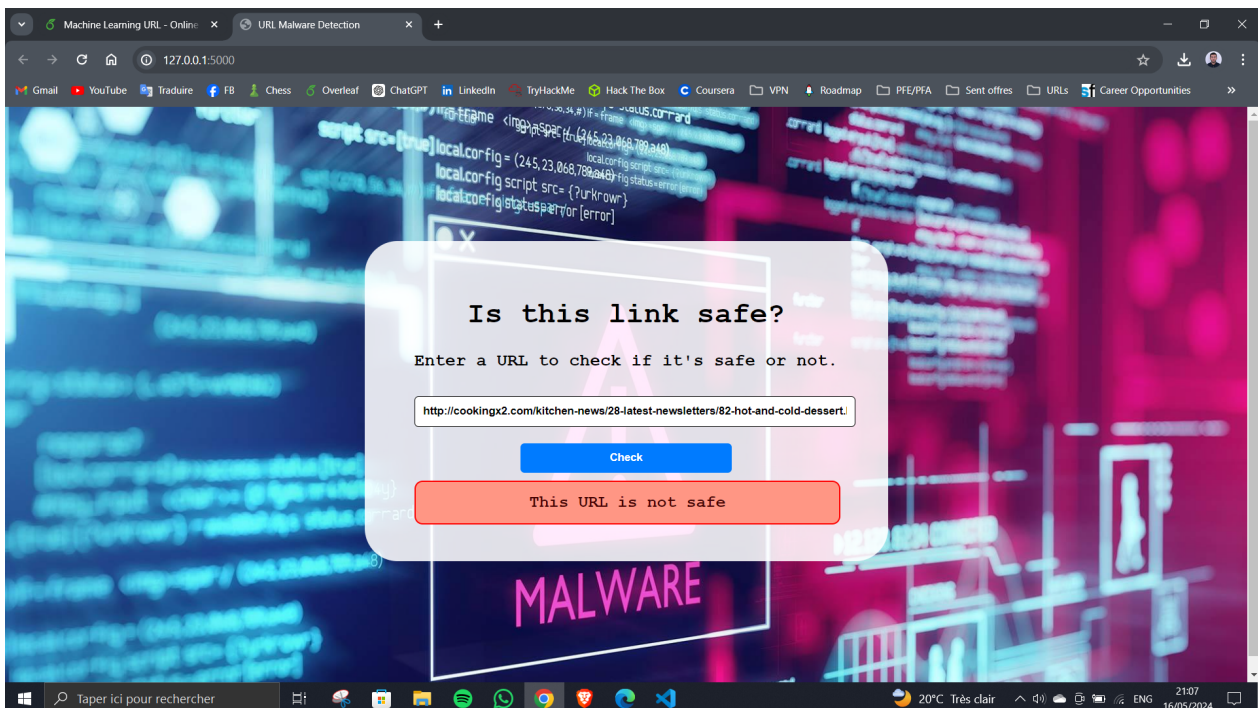
Si l'utilisateur clique sur le bouton "*check*" sans saisir d'url, l'application demande de fournir une URL pour effectuer une prédiction



Lorsque l'utilisateur saisit une URL que le modèle identifie comme sûre, l'application répond avec un message indiquant que cette URL est sûre.



Lorsque l'utilisateur saisit une URL que le modèle identifie comme dangereuse, l'application répond avec un message indiquant que cette URL n'est pas sûre.



CONCLUSION

Dans cette étude, nous avons exploré l'utilisation de techniques de machine learning pour détecter les URL malveillantes. À travers différentes étapes telles que la collecte des données, le nettoyage, la transformation, et l'entraînement de modèles, nous avons développé un système capable d'identifier avec précision les URL potentiellement dangereuses.

Cependant, il convient de noter quelques limitations de cette étude. Tout d'abord, malgré nos efforts pour nettoyer et prétraiter les données, il reste possible que des biais ou des artefacts subsistent dans le jeu de données. De plus, la détection des URL malveillantes est un défi en constante évolution, et notre modèle peut ne pas être en mesure de détecter les attaques sophistiquées ou récentes. Enfin, la disponibilité de données d'entraînement étiquetées peut limiter la capacité de généralisation du modèle à de nouveaux contextes ou types d'attaques.

Pour surmonter ces limitations, des travaux futurs pourraient se concentrer sur l'exploration de nouvelles sources de données ou l'amélioration des techniques de prétraitement pour réduire les biais. De plus, l'intégration de l'apprentissage en continu et de la détection d'anomalies en temps réel pourrait permettre au système de s'adapter plus efficacement aux menaces émergentes.

Répertoire GitHub

Vous pouvez voir le code complet ici : [\[Lien\]](#)

Références

- ▶ Malicious URLs dataset [\[Lien\]](#)
- ▶ Comprendre ce qu'est une URL [\[Lien\]](#)
- ▶ Extracting Feature Vectors For Malicious URL Detection (By Good Robots , Towards Data Science) [\[Lien\]](#)
- ▶ Malicious URL Detection and Classification Analysis using Machine Learning Models (By Upendra Shetty , Anusha Patil and Mohana) [\[Lien\]](#)
- ▶ Machine Learning for Malicious URL Detection (By Gold Wejinya and Sajal Bhatia) [\[Lien\]](#)